Practical 7

Model a simple Bayesian network or probabilistic classifier for prediction or diagnosis and perform inference using a suitable Python library.

Objective

To model a small medical-diagnosis Bayesian Network (Flu, Cold, Fever, Fatigue, Cough) using the pgmpy library, define its conditional probability tables (CPDs), and perform exact probabilistic inference (marginal, conditional and MAP queries) using Variable Elimination.

Procedure

1. Install and import pgmpy, and define the network structure as a Directed Acyclic Graph: Flu -> Fever, Flu -> Fatigue, Cold -> Fever, Cold -> Cough.
2. Define TabularCPDs for each node: the prior probabilities of Flu and Cold, and the conditional probabilities of Fatigue | Flu, Cough | Cold, and Fever | Flu, Cold.
3. Attach all CPDs to the DiscreteBayesianNetwork model and validate the model structure with check_model().
4. Create a VariableElimination inference object over the model for exact inference.
5. Query P(Flu | Fever=yes, Cough=yes) to see how evidence updates belief in Flu.
6. Query P(Cold | Fever=yes, Fatigue=no) to see how evidence updates belief in Cold.
7. Query the marginal P(Fever) with no evidence, and finally compute the MAP (most probable) joint assignment of Flu and Cold given Fever=yes, Fatigue=yes, Cough=no.

Program / Code

```
# ================================================================
# PRACTICAL 7
# Bayesian Network for Diagnosis - built and queried using pgmpy
# ================================================================

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# ----------------------------------------------------------------
# Network structure:
#   Flu -> Fever
#   Flu -> Fatigue
#   Cold -> Cough
#   Cold -> Fever
# A patient's Fever can be caused by Flu and/or Cold; Fatigue by Flu;
# Cough by Cold.
# ----------------------------------------------------------------

model = DiscreteBayesianNetwork([
```

```
    ("Flu", "Fever"),
    ("Flu", "Fatigue"),
    ("Cold", "Cough"),
    ("Cold", "Fever"),
])

cpd_flu = TabularCPD(variable="Flu", variable_card=2, values=[[0.90], [0.10]])
cpd_cold = TabularCPD(variable="Cold", variable_card=2, values=[[0.80], [0.20]])

cpd_fatigue = TabularCPD(
    variable="Fatigue", variable_card=2,
    values=[[0.90, 0.20],
            [0.10, 0.80]],
    evidence=["Flu"], evidence_card=[2],
)

cpd_cough = TabularCPD(
    variable="Cough", variable_card=2,
    values=[[0.85, 0.30],
            [0.15, 0.70]],
    evidence=["Cold"], evidence_card=[2],
)

# Fever depends on both Flu and Cold: P(Fever=1 | Flu, Cold)
cpd_fever = TabularCPD(
    variable="Fever", variable_card=2,
    values=[
        [0.95, 0.20, 0.30, 0.02],   # Fever = 0 (no)
        [0.05, 0.80, 0.70, 0.98],   # Fever = 1 (yes)
    ],
    evidence=["Flu", "Cold"], evidence_card=[2, 2],
)

model.add_cpds(cpd_flu, cpd_cold, cpd_fatigue, cpd_cough, cpd_fever)

assert model.check_model()

print("=" * 65)
print("     BAYESIAN NETWORK FOR DIAGNOSIS  (built with pgmpy)")
print("=" * 65)
print("\nNetwork edges:")
for edge in model.edges():
    print(f"   {edge[0]}  -->  {edge[1]}")

infer = VariableElimination(model)

print("\n" + "-" * 65)
print("QUERY 1 : P(Flu | Fever = yes, Cough = yes)")
q1 = infer.query(variables=["Flu"], evidence={"Fever": 1, "Cough": 1}, show_progress=False)
print(q1)

print("-" * 65)
print("QUERY 2 : P(Cold | Fever = yes, Fatigue = no)")
q2 = infer.query(variables=["Cold"], evidence={"Fever": 1, "Fatigue": 0}, show_progress=False)
print(q2)

print("-" * 65)
```

```
print("QUERY 3 : P(Fever) -- marginal, no evidence")
q3 = infer.query(variables=["Fever"], show_progress=False)
print(q3)

print("-" * 65)
print("QUERY 4 : Most probable Flu/Cold given Fever=yes, Fatigue=yes, Cough=no")
map_result = infer.map_query(
    variables=["Flu", "Cold"],
    evidence={"Fever": 1, "Fatigue": 1, "Cough": 0},
    show_progress=False,
)
print(map_result)
print("=" * 65)
```

Output

```
================================================================
     BAYESIAN NETWORK FOR DIAGNOSIS  (built with pgmpy)
================================================================

Network edges:
   Flu  -->  Fever
   Flu  -->  Fatigue
   Cold  -->  Cough
   Cold  -->  Fever

----------------------------------------------------------------
QUERY 1 : P(Flu | Fever = yes, Cough = yes)
+--------+------------+
| Flu    |   phi(Flu) |
+========+============+
| Flu(0) |     0.8276 |
+--------+------------+
| Flu(1) |     0.1724 |
+--------+------------+
----------------------------------------------------------------
QUERY 2 : P(Cold | Fever = yes, Fatigue = no)
+---------+-------------+
| Cold    |   phi(Cold) |
+=========+=============+
| Cold(0) |      0.2462 |
+---------+-------------+
| Cold(1) |      0.7538 |
+---------+-------------+
----------------------------------------------------------------
QUERY 3 : P(Fever) -- marginal, no evidence
+----------+--------------+
| Fever    |   phi(Fever) |
+==========+==============+
| Fever(0) |       0.7444 |
+----------+--------------+
| Fever(1) |       0.2556 |
+----------+--------------+
----------------------------------------------------------------
QUERY 4 : Most probable Flu/Cold given Fever=yes, Fatigue=yes, Cough=no
{'Flu': 1, 'Cold': 0}
```

==========

Conclusion

A 5-node Bayesian Network for symptom-based diagnosis was built and queried using pgmpy's exact Variable Elimination algorithm. Given Fever and Cough as evidence, P(Flu) rose only modestly to about 0.17 because Cough is explained mainly by Cold, not Flu — showing the network correctly performing 'explaining away' between the two competing causes of Fever. Conversely, given Fever with no Fatigue, P(Cold) rose to about 0.75, correctly favouring Cold as the more likely cause since Fatigue (a Flu symptom) was absent. The MAP query correctly identified Flu=yes, Cold=no as the most probable joint diagnosis for the fatigue+fever-without-cough evidence pattern. This confirms that a Bayesian Network, once its structure and CPDs are specified, allows principled probabilistic diagnosis and reasoning under uncertainty far more flexibly than a hand-written rule-based system.